EpiScope: A Big Data Analytics Pipeline for Synthetic Disease Surveillance, Outbreak Detection, and Preparedness Support

Md. Samin Osman (0222220005101042)
Robiul Hassan (0222220005101043)
Walid Talal (0222220005101045)
Md. Mahafujul Alam (0222220005101066)
CSE 4345 - Big Data Analytics, Spring 2026
Department of Computer Science and Engineering
Premier University, Chattogram, Bangladesh
Group Name: Team Odyssey
Repository: https://github.com/MDRobiulhassan/EpiScope
Presentation video: https://youtu.be/553MRqFQOEM

***Abstract*—**Public-health analytics can require the processing of large encounter datasets, data-quality controls, trend analysis, and the preparation of information for operational planning. This paper presents EpiScope, a prototype big-data pipeline that demonstrates these tasks using synthetic disease-encounter data. The architecture uses Apache Spark for data transformations, Bronze, Silver, and Gold processing layers, Parquet analytical storage, baseline outbreak detection and forecasting, resource-allocation and preparedness-calendar modules, and a D3.js dashboard export path. In a documented 25-million-row run, 24,825,210 records were retained in the clean Silver output and 174,790 were quarantined, reconciling to the input count. Documented processing-stage timings total 27.90 minutes, excluding synthetic-data generation. The latest saved detection evaluation reports 190 true positives, 25,336 false positives, 169 false negatives, and 554,145 true negatives, corresponding to 52.92% sensitivity, 4.37% false-positive rate, and 0.74% precision. The latest saved 28-day forecast evaluation reports a mean absolute error of 2.0182 and a root mean squared error of 2.6986 across 8,960 district-disease-day observations. Results are limited to the synthetic dataset and current evaluation procedures. Historical metrics differ substantially from the latest saved evaluations; their original run configurations are unavailable. EpiScope is therefore presented as an educational and analytical prototype, not a validated public-health decision system.

***Index Terms*—big data analytics, Apache Spark, data quality, disease surveillance, outbreak detection, forecasting, privacy-aware visualization**

I. Introduction

Disease-surveillance workflows may involve large volumes of records from different sources and require reliable preparation before analytical results can be presented. A useful analytics prototype should demonstrate data ingestion, validation, aggregation, analytical processing, and the controlled publication of derived information.

EpiScope was developed as a big-data analytics project to demonstrate such a workflow on synthetic encounter records. It combines layered data processing with baseline detection and forecasting, resource-allocation and preparedness-calendar outputs, and a browser dashboard. The project emphasizes an end-to-end data path and records limitations in the evaluation and privacy checks.

The main contributions of this project are:

- A Bronze, Silver, and Gold processing workflow for synthetic encounter data.
- A documented large-input data-quality reconciliation and quarantine process.
- Baseline outbreak-detection and fixed-horizon forecasting evaluations.
- Experiments documenting pipeline timing, Gold-stage scaling, and Spark configuration comparisons.
- A public-data preparation step that suppresses tested small case counts before dashboard export.

The paper does not claim clinical validity, real-world epidemiological accuracy, or formal privacy guarantees.

A. CEP Learning Attributes P1–P7

The brief expects evidence of distributed processing, columnar storage, data integration and quality, surveillance time-series analysis, optimization, visual design, and health-data privacy (P1). It emphasizes competing stakeholder needs and constraints (P2), justified and tested choices (P3), imperfect data including late, missing, duplicated, skewed, or artifactual reports (P4), combining written guidance and judgement (P5), stakeholder needs (P6), and end-to-end dependency awareness (P7). Spark, Parquet, validation, analytics, optimization, and dashboard components address parts of P1 and P7. Synthetic-only inputs, limited artifact simulation, modest evaluation,

and incomplete stakeholder testing leave substantial evidence gaps. Resource constraints, interaction, innovation, societal consequences, and unfamiliar tasks (A1–A5) are discussed as context, not measured outcomes.

B. CEP Design Conflicts C1–C5

- **C1, sensitivity versus specificity:** the detector threshold is a prototype choice; saved precision is low. No threshold study demonstrates an acceptable alert workload.
- **C2, geographic detail versus privacy:** district aggregates and suppression of counts 1–4 are used; complementary suppression and cross-view inference protection are not established.
- **C3, efficiency versus equity:** the optimizer applies a minimum district allocation, but does not use population denominators or demonstrate a measured efficiency–equity frontier.
- **C4, freshness versus cost and complexity:** daily inventory replay is separate from analytics; corrected snapshots are not merged and no cost study is recorded.
- **C5, accuracy versus interpretability:** transparent statistical baselines are used, but forecast accuracy and detector precision are limited; no validated model comparison resolves this conflict.

The report includes a project evidence summary, a four-member contribution matrix, member-specific learning reflections, and CEP requirement traceability in its appendices. The separate two-page policy brief accompanies the technical report as a standalone submission document.

II. System Architecture

EpiScope separates data preparation, analytics, and dashboard serving. Synthetic encounter records are ingested into the Bronze layer. The Silver stage validates records and separates accepted rows from quarantined rows. The Gold stage creates aggregated analytical datasets. Detection, forecasting, resource allocation, and preparedness-calendar modules consume analytical outputs. Public-data preparation is then applied before JSON datasets are exported to the D3.js dashboard.

Apache Spark is used for data transformations, Parquet for processed analytical storage, and D3.js v7 for browser visualization [1]–[3]. The project runs in a Google Colab environment with Spark in local mode for the recorded experiments. Consequently, measured runtimes depend on the runtime allocation, storage, partitioning, caching, and Spark configuration.

III. Data and Processing Method

A. Synthetic Input

The encounter records and outbreak labels used in the recorded experiments are synthetic. The project includes a generator for creating encounter records at different scales. Weather-related fields, if present in a given generated schema, must likewise be treated as synthetic or derived unless their provenance is independently established. Results should not be interpreted as evidence of performance on hospital records, government surveillance data, or real weather observations.

B. Layered Processing

The pipeline uses three logical data layers:

- **Bronze:** ingestion of source encounter records.
- **Silver:** validation, cleaning, and quarantine of records that fail implemented checks.
- **Gold:** aggregated datasets for downstream analytics and serving.

The Silver stage aims to preserve accounting between accepted and quarantined rows. Reconciliation demonstrates count consistency but does not independently prove that every validation rule is correct.

C. Incremental Inventory Input

The project includes a separate synthetic daily inventory feed with facility stock and bed-occupancy fields. The Colab workflow generates a one-day snapshot and processes it through Bronze, Silver, and Gold. Bronze appends only unseen `(snapshot_date, facility_id)` keys, so replaying the same snapshot should append zero rows. The first-write-wins key policy does not apply corrections to a previously ingested facility-day. This incremental path is currently a parallel demonstration; the resource-allocation optimizer does not consume its Gold output. The submitted Colab notebook records a successful two-pass execution of the inventory pipeline.

D. Analytics

The outbreak detector is a prototype evaluated at the district-disease-day level for observations with at least 14 baseline days. Forecasting uses a seven-day-average baseline computed before the test period and repeated as a fixed prediction across a 28-day horizon. This is not a rolling forecast that updates predictions every day.

Resource allocation and preparedness-calendar components are included as analytical demonstrations; their outputs are not validated operational guidance.

E. Public Data and Privacy

A public-data preparation step precedes dashboard JSON export. The documented policy suppresses exact case counts from one through four. A recorded check of the public case export found no exposed values in that range in the tested output.

This is a limited check, not a complete privacy audit: inference may still be possible through geographic and temporal combinations, totals, linked views, or repeated queries.

IV. Experimental Setup

The recorded experiments were run in Google Colab using Spark in local mode. The documented full-scale processing input contained 25,000,000 rows. Synthetic-data generation is excluded from the reported 27.90-minute processing-stage total.

TABLE I
DOCUMENTED SILVER-STAGE RECONCILIATION

Measure	Rows
Input	25,000,000
Clean output	24,825,210
Quarantined output	174,790
Clean + quarantined	25,000,000
Quarantine rate	0.69916%

TABLE II
DOCUMENTED PROCESSING-STAGE RUNTIME

Stage	Runtime (min)
Bronze	5.16
Silver	12.83
Gold	3.57
Outbreak detection	1.47
Forecast generation	0.94
Resource allocation	0.59
Preparedness calendar	0.57
Public-data build	0.50
Dashboard export	0.66
Forecast evaluation	0.86
Detection evaluation	0.75
Total measured stages	27.90

Forecast evaluation used the test period from December 4 through December 31, 2025, with 8,960 district-disease-day observations. Detection metrics are taken from the latest saved evaluation output. Timing, optimization, scaling, and data-quality counts are documented historical measurements; raw timing logs were not available in the audited paths.

V. RESULTS

A. Data Quality

Table I summarizes the documented reconciliation for the 25-million-row run.

The clean and quarantined counts sum to the input count. These values were transcribed from the project's documentation and were not independently recounted during report preparation.

B. Pipeline Runtime

Table II presents documented stage runtimes. The total is a sum of recorded stages, not an independent timing measurement.

The documented total is below the project's 45-minute target. Synthetic-data generation, recorded separately at approximately 7.17 minutes, is excluded. These measurements are environment-specific, and the raw timing logs were not found in the audited paths.

C. Outbreak Detection

Table III shows the latest saved detection evaluation.

Sensitivity indicates that the detector identifies approximately half of the positive evaluation cases. However, precision of 0.74% means that false-positive alerts greatly outnumber true-positive alerts. This substantially limits the usefulness of the current detector without review and further tuning. It must not be used for unreviewed operational decisions.

TABLE III
LATEST SAVED OUTBREAK-DETECTION EVALUATION

Metric	Value
True positives (TP)	190
False positives (FP)	25,336
False negatives (FN)	169
True negatives (TN)	554,145
Sensitivity	52.92%
False-positive rate	4.37%
Precision	0.74%

D. Forecasting

The latest saved forecast evaluation contains 8,960 observations for December 4 to 31, 2025. The seven-day-average fixed-horizon baseline achieved MAE $= 2.0182$ cases and RMSE $= 2.6986$ cases. MAE expresses the average absolute difference between predicted and actual counts; RMSE weights larger errors more heavily. These values describe only the current synthetic evaluation and depend on the evaluation target and calendar-completion procedure.

Earlier project documentation reported MAE approximately 5.9894 and RMSE approximately 11.6049. Their source run and configuration are unavailable for independent verification, so these figures are identified as historical. The latest saved evaluation is reported separately as the current result.

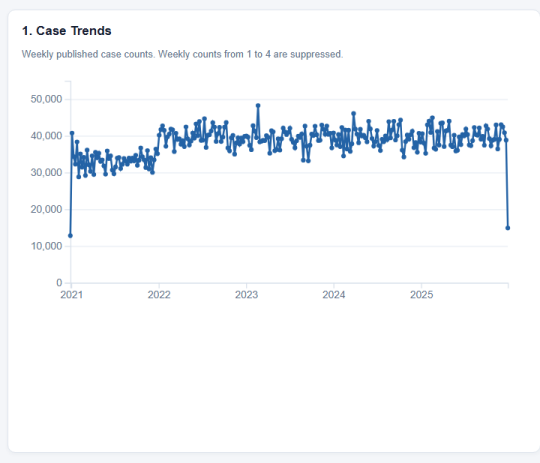
E. Optimization and Scaling

Documented Gold-stage experiments report 4.91 minutes for AQE disabled with 200 shuffle partitions, compared with 3.45 minutes for AQE enabled with 32 partitions. The latter is approximately 29.7% lower than the 4.91-minute baseline. Because these are historical single-run measurements without raw logs in the audited paths, they should be interpreted as indicative observations rather than a reliable general speedup.

Gold-stage runtimes were documented as 1.55 minutes for 6.25 million rows, 2.03 minutes for 12.5 million rows, and 3.45 minutes for 24,825,210 clean rows. The smaller datasets were generated using prefix or row-limit samples and may not preserve the full dataset distribution. These measurements describe the Gold stage only, not end-to-end pipeline scaling.

F. Dashboard Export Snapshot

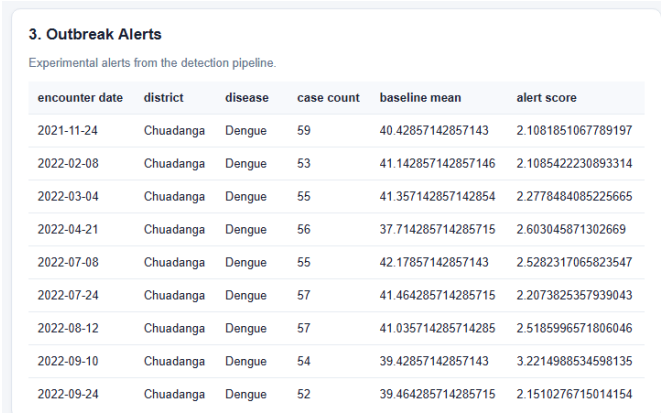
The saved dashboard files contain 36,000 weekly case rows, 17,063 alerts, 8,960 forecasts, 64 district allocations totaling 100 units, and 3,840 preparedness-calendar rows. The combined JSON payload is 9.81 MiB. The weekly case counts range from 32 to 3,667, with no values from 1 through 4 in this saved file. These counts were read from the local JSON exports. The cases and alerts files have later modification times than the forecast, resource, and calendar files, so the payload is a mixed saved snapshot rather than a synchronized pipeline export.



(a) Weekly case trends



(b) District comparison

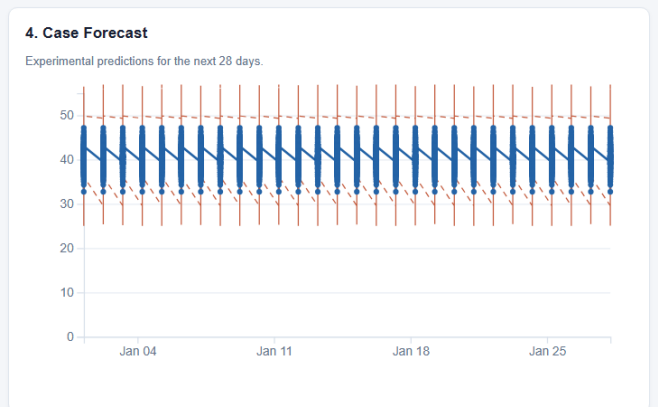


3. Outbreak Alerts

Experimental alerts from the detection pipeline.

encounter date	district	disease	case count	baseline mean	alert score
2021-11-24	Chuadanga	Dengue	59	40.42857142857143	2.1081851067789197
2022-02-08	Chuadanga	Dengue	53	41.142857142857146	2.1085422230893314
2022-03-04	Chuadanga	Dengue	55	41.357142857142854	2.2778484085225665
2022-04-21	Chuadanga	Dengue	56	37.714285714285715	2.603045871302669
2022-07-08	Chuadanga	Dengue	55	42.17857142857143	2.5282317065823547
2022-07-24	Chuadanga	Dengue	57	41.464285714285715	2.2073825357939043
2022-08-12	Chuadanga	Dengue	57	41.035714285714285	2.5185996571806046
2022-09-10	Chuadanga	Dengue	54	39.42857142857143	3.2214988534598135
2022-09-24	Chuadanga	Dengue	52	39.464285714285715	2.1510276715014154

(c) Outbreak alerts



(d) 28-day case forecast

Fig. 1. Dashboard views: (a) weekly published case counts, with counts from 1 through 4 suppressed; (b) district ranking by published case totals; (c) experimental alert table; and (d) experimental 28-day forecast. The dashboard uses synthetic data.

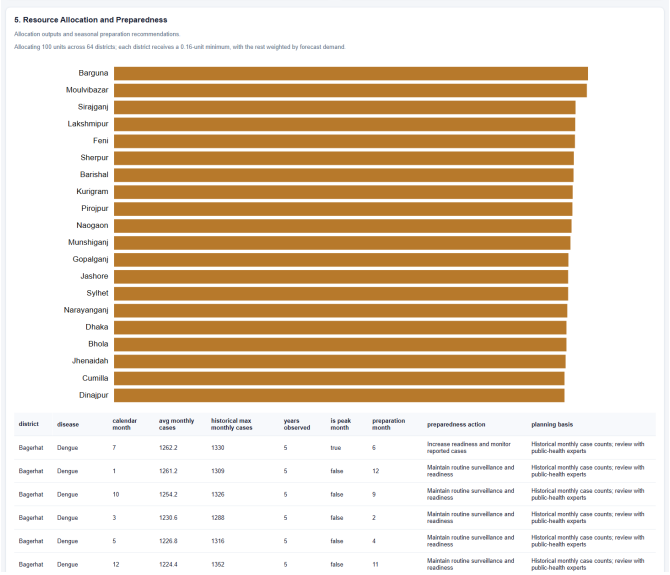


Fig. 2. Illustrative resource allocation and historical preparedness calendar. The allocation view shows a 100-unit budget with a 1/64-unit minimum per district and remaining units weighted by forecast demand. Calendar rows summarize historical district-disease-month patterns.

G. Dashboard Views and Interpretation

Figures 1 and 2 show the local dashboard at its default all-district, all-disease filters, using saved public JSON exports. They document the interface state and are not independent validation of the underlying analytics.

The district panel (Fig. 1b) ranks published case totals (the leading displayed district is Gazipur at 166,660); it is not a population-normalized risk comparison. Alert rows (Fig. 1c) are experimental signals and must be considered alongside the saved evaluation's 0.74% precision. The forecast (Fig. 1d) is a simple 28-day baseline; interval coverage has not been measured. The allocation (Fig. 2) does not account for current stock or population denominators. Calendar entries are planning prompts, not validated next-year recommendations. All captures are static views of saved public exports.

VI. Discussion

The project demonstrates an integrated workflow that connects large-input processing, data-quality reconciliation, baseline analytics, and dashboard-oriented export. The documented Silver-stage reconciliation provides evidence of row accounting for the recorded run. The runtime records suggest that the pipeline stages fit within the stated processing target in the measured environment, although the absence of raw timing logs limits independent verification.

The analytical results also show why evaluation metrics must be interpreted together. A sensitivity of 52.92% may appear useful in isolation, but precision of 0.74% indicates a high false-alert burden. Similarly, the forecast errors cannot establish real-world predictive utility because the evaluation data are synthetic and the baseline is intentionally simple. The difference between current and historical forecast metrics highlights the importance of experiment provenance and automated result capture.

VII. Limitations and Future Work

The findings should be interpreted within the scope of the prototype and its saved experiments. Encounter records and evaluation labels are synthetic, so the reported results do not establish performance on real surveillance data or represent real disease incidence. The Spark runs were performed in local mode in Google Colab; their timings depend on the allocated runtime, storage behavior, partitioning, caching, and configuration. The 25-million-row reconciliation demonstrates that the recorded clean and quarantined counts sum to the input, but it does not by itself establish the correctness of every validation rule. Per-rule outcome counts and independent rule-level validation are not part of the reported reconciliation.

The detection evaluation reports 52.92% sensitivity and 0.74% precision. The false-positive burden is therefore substantial, and the detector is not suitable for unreviewed alerting. Evaluation is based on the current synthetic labels and district-disease-day procedure; the saved metrics do not establish performance across districts, years, or real outbreak conditions. Forecast evaluation likewise uses synthetic observations and a fixed seven-day-average baseline repeated over 28 days. Its MAE and RMSE do not demonstrate performance against seasonal or operational forecasting alternatives, and prediction-interval coverage has not been evaluated. Earlier saved forecast metrics differ from the latest evaluation, and their original run configurations are unavailable, limiting direct comparison.

Resource allocation is an illustrative optimization over abstract units. It does not incorporate facility-level stock, population denominators, transport constraints, or a measured comparison with a population-proportional baseline. The preparedness calendar summarizes historical synthetic patterns and should be treated as a planning aid rather than a validated forecast. The inventory feed is a separate synthetic demonstration: replay protection is keyed by facility and snapshot date, while corrected snapshots are not merged into prior records and the optimizer does not consume the inventory output.

The public dashboard suppresses published case counts from one through four in the tested export. This direct-count check is not a formal privacy guarantee and does not rule out disclosure through linked time periods, geographic totals, repeated queries, or other combinations. The saved dashboard files also have different modification times, so their contents represent a mixed snapshot rather than one synchronized export. Finally, recorded runtime and configuration comparisons are historical measurements; the raw timing logs are unavailable in the audited paths, which limits independent reproduction.

Further development should prioritize evaluation that addresses these limits: validate the pipeline against representative, appropriately governed data; record versioned inputs, configurations, and stage-level logs; report validation outcomes per rule; compare detection thresholds and methods using district-year false-alert rates and detection delay; and evaluate rolling forecasts against seasonal-naive and other suitable baselines, including interval coverage. Allocation experiments should incorporate population and stock constraints and compare alternatives under matched budgets. Privacy testing should examine linked outputs and query combinations, and the inventory path should define how corrected snapshots affect downstream allocations. These steps would provide stronger evidence before any use beyond education and research; domain and governance review would be necessary for real public-health application.

VIII. Conclusion

EpiScope demonstrates a layered big-data pipeline for synthetic disease-surveillance analytics, including data cleaning and quarantine, aggregation, baseline detection and forecasting, and dashboard data preparation. The documented large-input run reconciles 25 million input rows and reports 27.90 minutes across measured processing and evaluation stages, excluding data generation. The latest saved evaluations report 52.92% detection sensitivity but only 0.74% precision, and forecast MAE/RMSE of 2.0182/2.6986 on a synthetic 28-day test period.

These results are preliminary and have limitations in historical comparability and independent reproducibility. The system should be regarded as an educational prototype rather than a validated public-health decision-support tool.

REFERENCES

[1] Apache Software Foundation, "Apache Spark Documentation." [Online]. Available: https://spark.apache.org/docs/latest/

[2] Apache Software Foundation, "Apache Parquet Documentation." [Online]. Available: https://parquet.apache.org/docs/

[3] M. Bostock, V. Ogievetsky, and J. Heer, "D^3: Data-Driven Documents," *IEEE Transactions on Visualization and Computer Graphics*, vol. 17, no. 12, pp. 2301–2309, Dec. 2011, doi: 10.1109/TVCG.2011.185.

[4] IEEE, "IEEE Reference Guide." [Online]. Available: https://journals.ieeeauthorcenter.ieee.org/your-role-in-article-production/ieee-editorial-style-manual/

Evidence and Evaluation Context

The following tables summarize recorded project outputs, repository contribution evidence, and the implementation scope represented by the CEP requirement identifiers. Performance figures describe saved experiments, with synthetic-data and evaluation limitations stated alongside the results.

TABLE IV
Project Evidence and Evaluation Scope

Area	Project evidence	Interpretation and scope
Scale and runtime	The documented encounter run processed 25,000,000 rows, retaining 24,825,210 clean rows and quarantining 174,790. Recorded processing and evaluation stages total 27.90 minutes; generation took 7.17 minutes separately.	These are recorded Google Colab measurements. Raw timing logs are unavailable, so the figures describe the recorded run.
Quality and layers	Bronze, Silver, and Gold processing; 15 named Silver checks; clean and quarantine counts reconcile exactly to the input.	The row totals document reconciliation. Per-rule failure counts may overlap and are not a count of unique rejected rows.
Detection	Saved evaluation: 190 true positives, 25,336 false positives, 169 false negatives, and 554,145 true negatives. Sensitivity is 52.92%, false-positive rate 4.37%, and precision 0.74%.	Evaluation uses synthetic labels at district-disease-day level after a 14-day baseline. The low precision indicates a high review burden.
Forecasting	A seven-day-average baseline is repeated for a fixed 28-day horizon. The saved evaluation has 8,960 observations, MAE 2.0182, and RMSE 2.6986.	Results use synthetic data from December 4 to 31, 2025. The reported evaluation has no prediction intervals or seasonal-naive comparison.
Allocation	The integer optimizer allocates a 100-unit budget across 64 districts, applying an allocation floor and weighting the remainder by forecast demand.	Outputs are illustrative planning results. They do not account for population denominators or live inventory.
Spark optimization	The documented Gold-stage comparison reports 4.91 minutes with AQE disabled and 200 shuffle partitions, and 3.45 minutes with AQE enabled and 32 partitions.	These are historical single-run timings without raw logs; they are observations for the recorded environment.
Dashboard and privacy	Five dashboard exports contain 36,000 weekly cases, 17,063 alerts, 8,960 forecasts, 64 district allocations, and 3,840 calendar rows. Combined JSON size is 9.81 MiB.	Saved exports have different modification times and form a mixed snapshot. The saved case file contains no weekly published counts from 1 through 4; this check is not a full disclosure-risk analysis.

Appendix A: Team Contribution Matrix

This matrix summarizes repository changes and member-reported contributions. Commit records identify repository activity and are not a measure of individual effort.

The table identifies repository changes and member-reported work. Commit history documents repository activity; it does not measure individual effort or assign contribution percentages.

TABLE V
REPOSITORY EVIDENCE FOR TEAM CONTRIBUTIONS

Member	Repository and contribution evidence	Work areas represented
Md. Samin Osman (0222220005101042)	Commit `e79403c` by `samin0sm` changed the generator, Silver validation, detection-threshold evaluation, forecasting, allocation, dashboard, tests, and run materials.	Data generation, quality validation, analytics, serving, tests, and execution materials.
Robiul Hassan (0222220005101043)	Commits under `Robiul Hassan` and `MDRobiulHassan` (same recorded email) include pipeline and inventory work, dashboard-data updates, and documentation; examples include `c382d73`, `059501d`, and `10fdd36`.	Pipeline and incremental feed, saved dashboard data, documentation, and integration.
Md. Mahafujul Alam (0222220005101066)	Commit `67fe413` records work on the presentation, report, documentation, and export materials.	Presentation and project-material preparation.
Walid Talal (0222220005101045)	Member-reported contribution: report writing and submission preparation.	Report writing and submission preparation.

APPENDIX B: INDIVIDUAL LEARNING REFLECTIONS

These reflections connect each member's documented project role with lessons evident in the implementation and evaluation. Repository activity supports the contribution summaries; it is not a measure of effort.

Md. Samin Osman (0222220005101042)

The recorded work spans data generation, Silver validation, analytics, dashboard changes, tests, and execution materials. Together, these components show that a data product depends on agreement between its generated inputs, quality rules, analytical methods, and serving logic. The saved results also demonstrate why implementation must be paired with evaluation: the detector has low precision, and the forecast history contains metrics from different runs that cannot be directly reconciled. This project experience supports a practical lesson for future analytics work: retain the input and configuration for every experiment, and connect reported metrics to the code and dataset version that produced them.

Robiul Hassan (0222220005101043)

The recorded work includes pipeline and inventory-feed changes, dashboard-data updates, and project documentation. The inventory path demonstrates the value of replay protection and also the limits of a first-write-wins policy when an earlier snapshot is corrected. The dashboard exports show a related integration concern: files produced at different times can form a mixed snapshot when viewed together. These implementation details highlight the need to define update behavior for incremental data and to make output freshness and provenance visible to users. Reliable analytics therefore depends on pipeline behavior and on clear communication of how and when each result was produced.

Walid Talal (0222220005101045)

The report and submission materials organize the project's implementation evidence and evaluation results. This work emphasizes the importance of distinguishing implemented components from claims supported by measured outputs. The differences between current and historical forecast metrics make that distinction especially important: a result is useful only when its evaluation period, method, and source artifacts are clear. Preparing the report also brings the system's limitations into view, including synthetic-only data, low detector precision, and the lack of a synchronized dashboard snapshot. Clear technical reporting should present these limitations alongside the design so readers can interpret the results at the right scope.

Md. Mahafujul Alam (0222220005101066)

The recorded project-material work includes the four-member presentation and supporting documentation. Preparing these materials highlights that communication is part of a technical system: a report or presentation needs to distinguish implemented behavior from experimental evidence and explain the limits of each result. This is particularly important for EpiScope because its data are synthetic and its alerts, forecasts, allocation, and preparedness outputs are experimental. Presenting the prototype with that context helps readers understand the dashboard as an educational demonstration rather than operational guidance.

APPENDIX C: CEP REQUIREMENT TRACEABILITY

This table maps CEP requirement identifiers to the project implementation and saved repository evidence. Scope notes identify synthetic demonstrations, recorded measurements, and the evaluation context represented by each artifact.

TABLE VI: Traceability to CEP requirement identifiers

CEP IDs	Implemented scope	Repository evidence and interpretation
ING-1 to 7	Synthetic encounter generation and Bronze-Silver-Gold batch processing.	`generator/generate_encounters.py`, the pipeline modules, and the Colab notebook document the 25-million-row synthetic run. The recorded dataset is synthetic.
DQ-1	Fifteen named Silver validation checks with quarantine reasons.	`pipeline/silver.py` applies the checks; `results/data_quality_metrics.json` records the saved row counts. A record can fail multiple checks.
DQ-2	Silver quality counts and reconciliation are recorded.	The saved input, clean, and quarantine totals reconcile to 25,000,000 rows.
DQ-3	Bronze ingestion records source-file and ingestion metadata for the batch input.	Lineage fields are present in the Bronze ingestion output; the report's recorded evidence is the saved pipeline output and its source code.
DQ-4	Encounter identifiers are checked for duplicate values; inventory replays use snapshot-date and facility keys.	The encounter duplicate check is defined in Silver. Inventory replay protection is demonstrated separately in `pipeline/inventory.py`.
PRV-1	Patient identifiers are salted-hashed before clean Silver output.	The hash salt is supplied through `EPISCOPE_HASH_SALT`; raw identifiers remain in restricted Bronze and quarantine data.
PRV-2	Exact public case counts from 1 through 4 are suppressed.	The saved weekly `cases.json` contains no published count in this interval. This is a direct-count check, not a complete inference analysis.
PRV-3 to 4	Public data are aggregated and small counts are suppressed.	The report makes no formal k-anonymity, differential-privacy, or role-based access-control claim.
AN-1 to 3	A rolling-baseline detector produces district-disease-day alerts and evaluation metrics.	The saved evaluation records 190 true positives, 25,336 false positives, 169 false negatives, and 554,145 true negatives.
AN-4 to 5	A seven-day-average baseline is repeated over a fixed 28-day forecast horizon.	The saved synthetic evaluation contains 8,960 observations, with MAE 2.0182 and RMSE 2.6986.
AN-6	Forecast case demand informs an illustrative resource-unit allocation.	Allocation is expressed as abstract units; no cases-to-beds, saline, or test-kit conversion is claimed.
AN-7 to 8	An integer optimizer distributes a 100-unit budget across 64 districts with a minimum allocation and forecast-weighted remainder.	The saved allocation output totals 100 units. It is an illustrative result, not a measured comparison against a population-proportional allocation.
AN-9	Preparedness-calendar output summarizes historical district-disease-month patterns and peak months.	Calendar records and actions are included in the saved dashboard export; they are planning prompts.
AN-10	The analytics presented in this report comprise detection, forecasting, allocation, and preparedness-calendar outputs.	No SEIR scenario results are included in the saved project outputs.
VIS-1 to 4	The D3.js dashboard presents case trends, district comparison, alerts, forecasts, and resource/preparedness views.	Five JSON exports total 9.81 MiB in the saved mixed snapshot. Their modification times differ.
VIS-5 to 7	The dashboard exposes budget, alert-threshold, and equity-floor planning controls.	Control values are illustrative. The report presents no usability-study or mobile-playback findings.
ENG-1	A pinned dependency file, Colab workflow, and batch entry script support execution.	The repository includes `requirements.txt`, `scripts/EpiScope_Colab_Run.ipynb`, and `scripts/run_pipeline.py`.
ENG-2	The Colab record documents the 25-million-row pipeline run and saved scaling measurements.	The full-scale run and smaller scaling experiments describe their recorded inputs and environment in `results/`.
ENG-3	AQE and shuffle-partition settings were compared in recorded Gold-stage experiments.	The reported comparison is a historical single-run measurement; raw Spark UI logs are not included in the saved evidence.
ENG-4	Validation tests and a two-pass incremental inventory demonstration are recorded.	The Colab test cell reports two passing tests in `tests/test_validation.py`; the inventory cell records initial ingestion and replay.